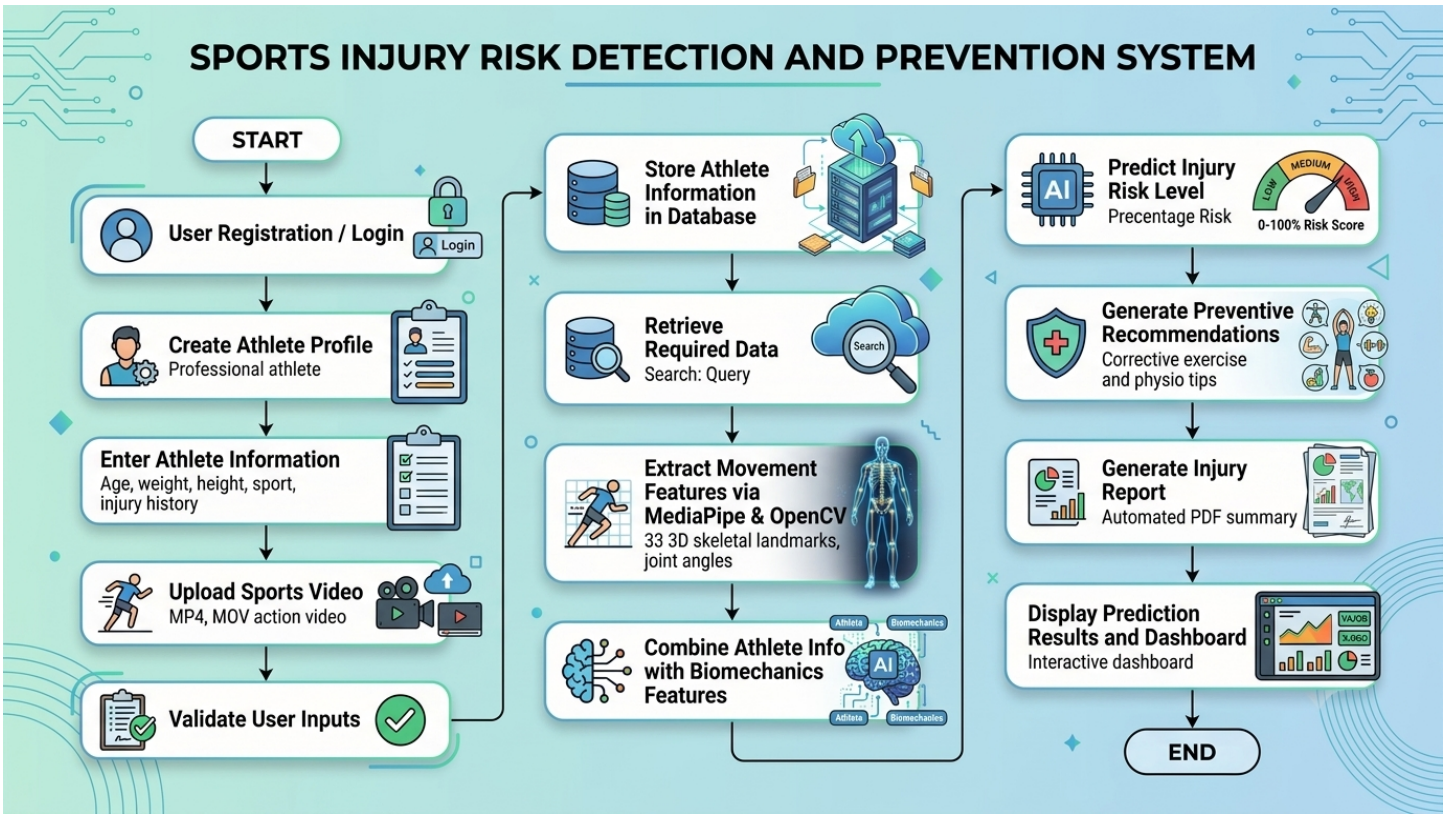
Sports Injury Risk Detection and Prevention System

Comprehensive Technical Architecture, Workflow & Future AI/ML Roadmap

1. Executive Summary

The **Sports Injury Risk Detection and Prevention System** is a non-invasive athletic screening platform that utilizes computer vision (MediaPipe & OpenCV) and biomechanical kinematics to analyze human movement patterns from video. It calculates joint angles, Range of Motion (ROM), and bilateral asymmetries to identify high-risk movement flaws (such as knee valgus collapse or excessive spinal lean) and deliver preventive exercise recommendations before non-contact injuries occur.

2. System Workflow Infographic



3. Technical Stack Breakdown

Component Layer	Technology	Role & Responsibility
Frontend UI	React.js 18 + Vite	Single Page Application (SPA) dashboard, athlete records & video upload interface.
Web Server	Nginx (Alpine)	High-performance static asset server with SPA routing (`try_files`) & API reverse proxy.
Backend API	FastAPI + Uvicorn	Asynchronous REST API, Pydantic data validation, and Swagger OpenAPI docs.
Pose Tracking AI	Google MediaPipe	Deep learning neural network extracting 33 3D skeletal landmarks per frame.
Video Processing	OpenCV (`cv2`)	Video decoding, frame sampling, visual skeleton overlay rendering.
Kinematics Math	NumPy & Pandas	3D vector trigonometry, joint angle computation, time-series landmark DataFrames.
Persistence Layer	SQLAlchemy + SQLite	Relational storage for user auth, athletes, movement data, and risk records.
Report Generator	ReportLab	Automated compilation of clinical PDF reports with observations and recommendations.
Containerization	Docker & Compose	Multi-container isolation with persistent storage volumes and health checks.

4. End-to-End Data Processing Pipeline

Step 1: User Authentication

Coaches/physios authenticate via POST /api/auth/register and /login. A secure session token is generated and attached as Authorization: Bearer <token> to all subsequent requests.

Step 2: Athlete Profile Management

Athletic baseline metrics (Age, Weight, Height, Sport, and Prior Injury History) are collected and saved in SQLite to provide context for kinematic evaluation.

Step 3: Video Ingestion & Validation

Action videos (MP4/MOV) are uploaded to /app/uploads with automated format, size, and header verification.

Step 4: MediaPipe 33-Landmark Pose Estimation

OpenCV decodes video frames; MediaPipe tracks 33 3D coordinates (x,y,z) per frame. An annotated video with skeletal overlays is rendered and saved to /app/results/.

Step 5: Kinematic Joint Calculations

Vector math computes Knee/Hip Range of Motion (ROM), Bilateral Asymmetry % (comparing left vs right leg loading), Trunk Lean Angle, and Movement Consistency across repetitions.

Step 6: Injury Risk Prediction & Reporting

Calculates a 0-100% Risk Score classified as **LOW**, **MEDIUM**, or **HIGH**, generates targeted corrective exercises, and compiles an exportable PDF report.

5. Future AI/ML Roadmap (Milestone 3)

In Milestone 3, the rule-based risk evaluation will be upgraded to a **Supervised Machine Learning & Deep Learning model**. The Milestone 2 feature extraction pipeline directly supplies the exact numerical features needed for ML training:

A. Machine Learning Feature Vector:

- Knee Symmetry Deficit (%): Left vs right knee angle deviation
- Hip Symmetry Deficit (%): Pelvic tilt and bilateral hip displacement
- Max Trunk Lean Angle (°): Correlates with spinal shear stress and core fatigue
- Movement Quality Score (0-100): Composite score of joint coordination
- Athlete Baseline (Age, Weight, Sport): Physical demographics
- Reported Injury History (0 or 1): Historical risk weighting

B. Recommended AI Frameworks & Architecture:

1. **Tabular Classification (XGBoost / Random Forest / LightGBM):** High accuracy and interpretability for vector-based injury classification.

2. **Temporal Deep Learning (PyTorch LSTM / 1D-CNN):** Directly models full sequence kinematics over the entire duration of a jump or sprint.

3. **Model Explainability (SHAP / LIME):** Quantifies exactly why a risk score was assigned (e.g. *'+35% risk attributed to 22° left knee valgus collapse'*).

C. Model Integration Architecture:

The model file (e.g., injury_model.pkl or .onnx) is placed in backend/app/models/ and loaded on server startup. The existing compute_risk() function calls model.predict_proba() seamlessly with zero breaking changes to frontend consumers.

6. Docker Deployment Architecture

The entire system is orchestratable via docker compose up --build. The frontend (Nginx on Port 5173/80) and backend (FastAPI on Port 8000) run in isolated containers connected via an internal Docker bridge network. Named volumes (sports_injury_backend_data, uploads, and results) guarantee persistent SQLite databases and video files across container lifecycles.